

Яковлев М. В.

Программирование беспилотного летательного аппарата Tello на языке Python

Учебное пособие



**Муниципальное общеобразовательное учреждение Иркутского районного
муниципального образования
«Средняя общеобразовательная школа поселка Молодежный»**

Яковлев М. В.

Программирование беспилотного летательного аппарата Tello на языке Python

Учебное пособие

пос. Молодежный, 2024

ОГЛАВЛЕНИЕ

ПРЕДИСЛОВИЕ.....	4
Техника безопасности при работе квадрокоптером:	4
Требования к месту полетов:	5
РАЗДЕЛ 1. ТЕОРИЕТИЧЕСКИЕ ОСНОВЫ И ИНСТРУКЦИЯ ПО УСТАНОВКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	6
1.1 Классификация беспилотных летательных аппаратов	6
1.2 Инструкция по установке кроссплатформенной среды PyCharm.....	8
1.3 Первый запуск Tello	16
РАЗДЕЛ 2. КОМАНДЫ УПРАВЛЕНИЯ И АЛГОРИТМИЧЕСКИЕ КОНСТРУКЦИИ	21
2.1 Команды управления полётом.....	21
2.2 Ввод данных, цикл while, оператор if и функции	27
2.3 Вложенные циклы	31
2.4 Угловое положение.....	33
2.5 Система визуального позиционирования.....	35

ПРЕДИСЛОВИЕ

Материал, представленный в учебном пособии, предлагается к изучению на третий год изучения информатики и главным образом отражает модуль «Программирование беспилотного летательного аппарата Tello на языке Python», который в свою очередь входит в рабочую программу по предмету информатика.

Учебное пособие содержит теоретические основы управления и классификацию беспилотных летательных аппаратов, инструкцию по установке программного обеспечения, а также серию задач на алгоритмические конструкции.

Есть известный факт, что люди, играя, гораздо лучше усваивают информацию, получают необходимые навыки и могут опробовать их на практике в безопасной игровой форме.

Идеальным инструментом для этого может послужить квадрокоптер Tello. Он и персональный компьютер представляют пример киберфизической системы. При этом часть вычислений будет брать на себя ПК, а часть — дрон, и работать они будут как единое целое

Техника безопасности при работе квадрокоптером:

- Тестирование и настройка аппарата должна производиться при снятых пропеллерах.
- Не дотрагивайтесь до движущихся частей моторов и винтов. Они вращаются с огромной скоростью и тяжёлые раны вам обеспечены.
- Когда аппарат находится в режиме «боевой готовности» не трогайте его, во избежание случайного включения двигателей.
- Не пытайтесь продолжать полет если батарея разряжена, иначе это может привести к неконтролируемому падению коптера.
- Не рекомендуется переключать коптер из режима стабилизации пока оператор не научится уверенно контролировать полет и справляться с базовыми маневрами
- При аварии либо нештатной ситуации во время посадки, когда не известна степень повреждения полетного контроллера необходимо:

- бросить полотенце на пропеллеры, так как они могут начать крутиться неожиданно
- сразу отключить питание
- Всегда проверяйте безопасное расстояние между коптером и зрителями.
- Убедитесь, что между вами и аппаратом нет людей
- Зрители всегда должны находится позади оператора
- Если кто-то нарушает зону безопасности - сажайте аппарат и подождите пока не освободится пространство для безопасного взлета.
- Убедитесь, что батарея не вставлена в коптер до тех пор, пока оператор не готов к полету.
- После приземления, отключить питание!
- Не используйте аппарат для осуществления действий, нарушающих законы Российской Федерации, не вмешивайтесь в частную жизнь граждан.

Требования к месту полетов:

1. НЕ используйте коптер в неблагоприятных погодных условиях.
2. Летайте только в местах, где вы можете держать коптер по крайней мере в 10 метрах от препятствий.
3. НЕ летайте на коптере по маршруту, который имеет резкое изменение уровня земли (например, внутри здания наружу), в противном случае функция позиционирования может быть нарушена, что влияет на безопасность полета.
4. Эксплуатация коптера и аккумулятора зависит от факторов окружающей среды
5. НЕ используйте коптер вблизи ЧП.
6. Во избежание помех между вашим интеллектуальным устройством и другим беспроводным оборудованием, *выключите другое беспроводное* оборудование во время полета на коптере.

РАЗДЕЛ 1. ТЕОРИЕТИЧЕСКИЕ ОСНОВЫ И ИНСТРУКЦИЯ ПО УСТАНОВКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

1.1 Классификация беспилотных летательных аппаратов

В Сети можно найти множество названий для него: дрон, квадрокоптер, коптер, мультикоптер, БЛА, БПЛА, беспилотник.

Дрон (drone) переводится с английского как «трутень». Это слово используется как синоним аббревиатуры БПЛА (или БЛА) — беспилотный летательный аппарат, представляющий собой воздушное судно без пилота, которое полностью дистанционно управляется из другого места: с земли, с борта воздушного судна либо запрограммировано и полностью автономно.

Беспилотник — это упрощённое название БПЛА.

Беспилотные летательные аппараты трудно классифицировать, так как они имеют очень разные характеристики, конфигурации и содержат множество компонентов.

Но общую классификацию провести можно (рис. 1). Среди БПЛА выделяют два типа: самолётный и вертолётный. Самым интересным для нас семейством последнего типа являются мультикоптеры.

Мультикоптер — летательный аппарат, построенный по вертолётной схеме, с тремя и более несущими винтами. Мультикоптеры также называют просто коптерами. Коптеры подразделяют по количеству пропеллеров: трикоптер — три пропеллера; квадрокоптер — четыре; гексакоптер — шесть; октокоптер — восемь пропеллеров.



Рис. 1. Одна из классификаций дронов

Расположение пропеллеров

На квадрокоптерах используются два типа двигателей: с вращением вала по часовой стрелке (CW — clockwise, рис. 2) и с вращением вала против часовой стрелки (CCW — counter clockwise). Схема установки моторов зависит от типа летательного аппарата. Каждый пропеллер (воздушный винт) приводится в движение собственным двигателем. Половина винтов вращается по часовой стрелке, половина — против.



Рис. 2. Пропеллеры CW для Tello

Для того чтобы совершить полет на квадрокоптере Tello, необходимо скачать и установить приложение Tello (рис. 3).

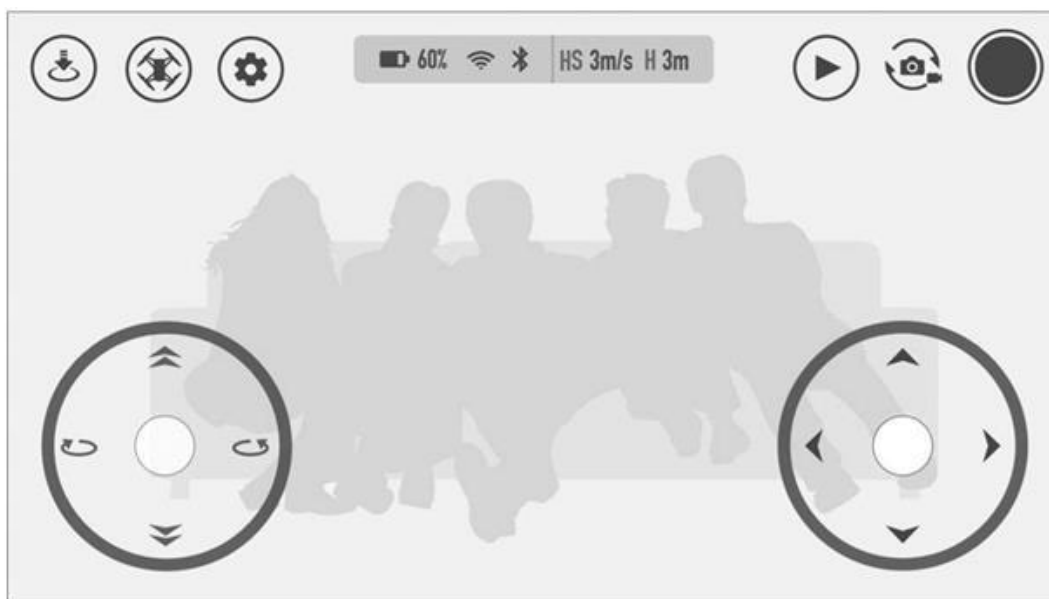


Рис. 3. Схема основного окна приложения Tello

1.2 Инструкция по установке кроссплатформенной среды PyCharm

Для написания программ для Tello, которые он сможет выполнять, нам потребуется Python (Пайтон, в русском языке распространено название Питон) — высокоуровневый язык программирования общего назначения.

Чтобы начать программировать на Python, необходимо скачать его с официального сайта python.org (рис. 4).

Далее нам понадобятся некоторые разновидности программ или технических средств, без которых программирование квадрокоптера будет невозможна.

И первое что от нас потребуется – это установить интерпретатор языка Python (рис. 5). Интерпретатор (от лат. *interpretator* — «толкователь») — это программа (разновидность транслятора), выполняющая построчный анализ, обработку и выполнение исходного кода программы — интерпретацию. Если все действия выполнены правильно, вы увидите окно завершения установки (рис. 5, 6).



Рис. 4. Страница сайта python.org



Рис. 5. Начало установки (установите указанный флажок)



Рис. 6. Установка прошла успешно

Для того чтобы эффективно связать наш квадрокоптер с языком программирования python нам потребуется IDE (Integrated Development Environment) — интегрированная среда разработки — комплекс программных средств, используемый программистами для разработки программного обеспечения.

Среда разработки включает в себя:

- текстовый редактор;
- транслятор, компилятор, интерпретатор;
- средства автоматизации сборки программы;
- отладчик.

Интегрированные среды разработки были созданы для того, чтобы максимизировать производительность программиста.

Для программирования нашего дрона будем использовать кроссплатформенную IDE PyCharm.

Нам интересна версия PyCharm Community Edition, так как именно она является общедоступной и бесплатной. Скачать её можно с сайта производителя: <https://www.jetbrains.com/pycharm/> (рис. 7,8).

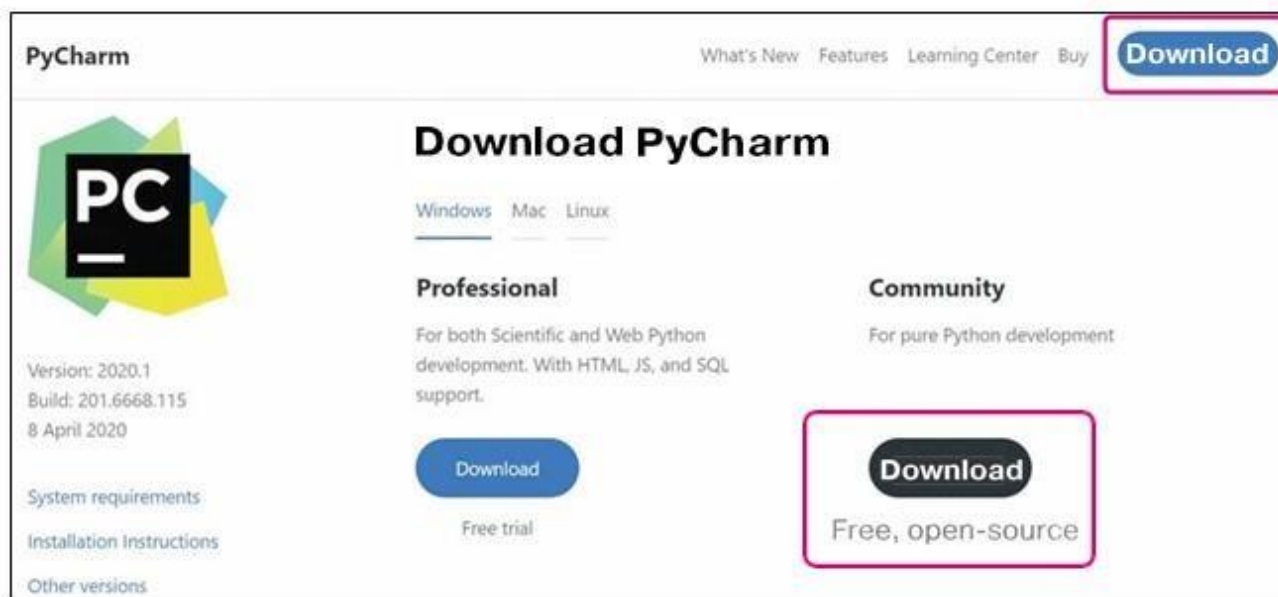


Рис. 7. Окно установщика PyCharm (необходимые опции)

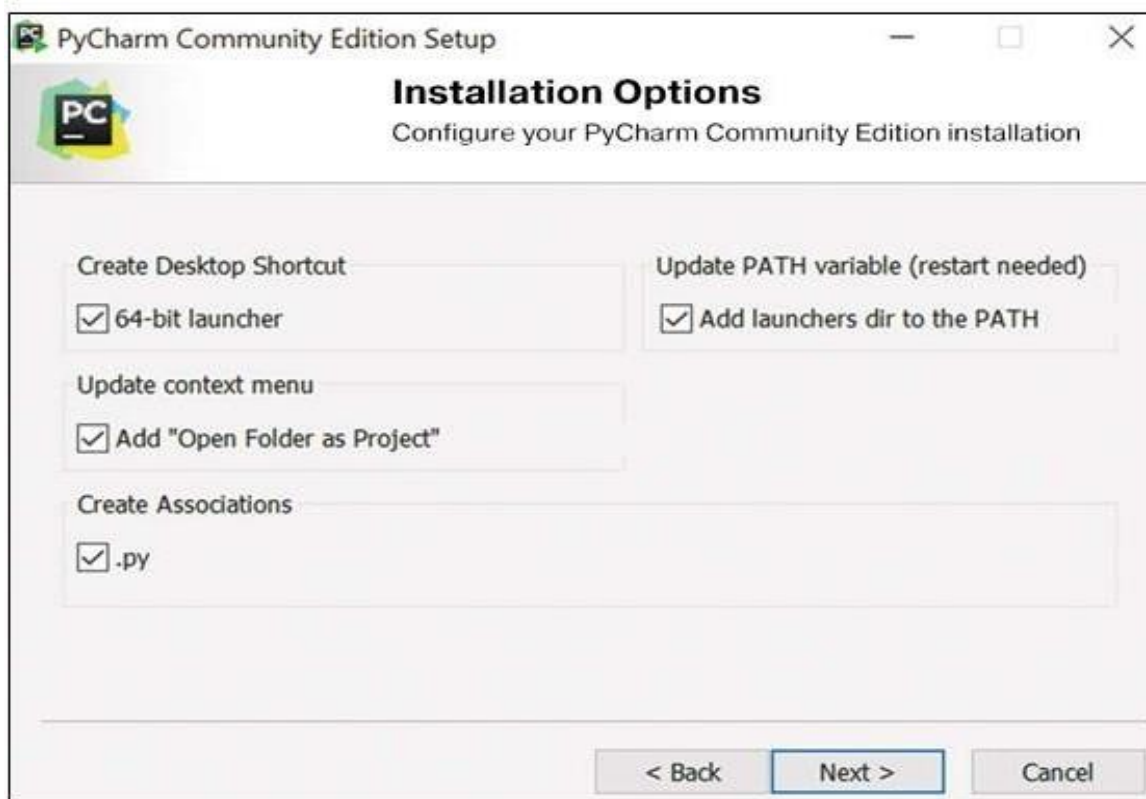


Рис. 8. Окно установщика PyCharm (необходимые опции)

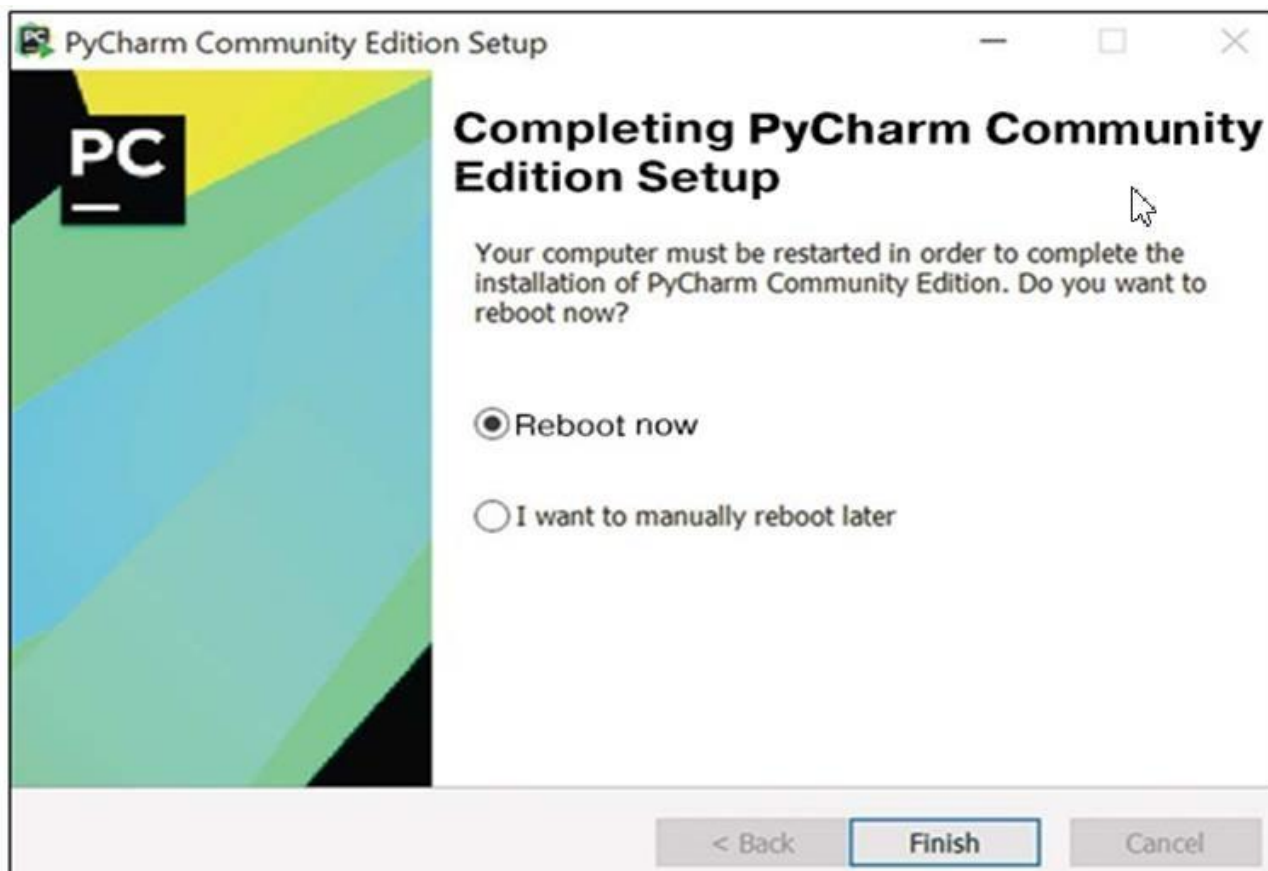


Рис. 9. Окно установщика PyCharm (необходимость перезагрузки)

Создайте в проекте файл, в котором будет писаться код программы (рис. 10, 11). Контекстное меню вызывается правой кнопкой мыши.

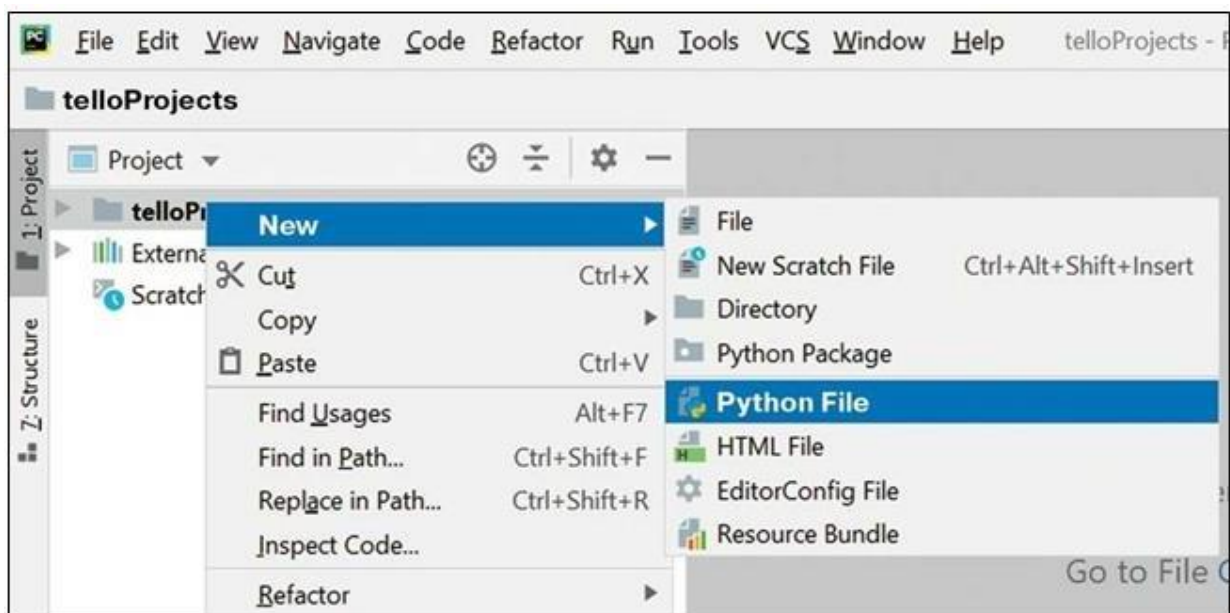


Рис. 10. Добавление файлов в проект (нам нужен Python File)

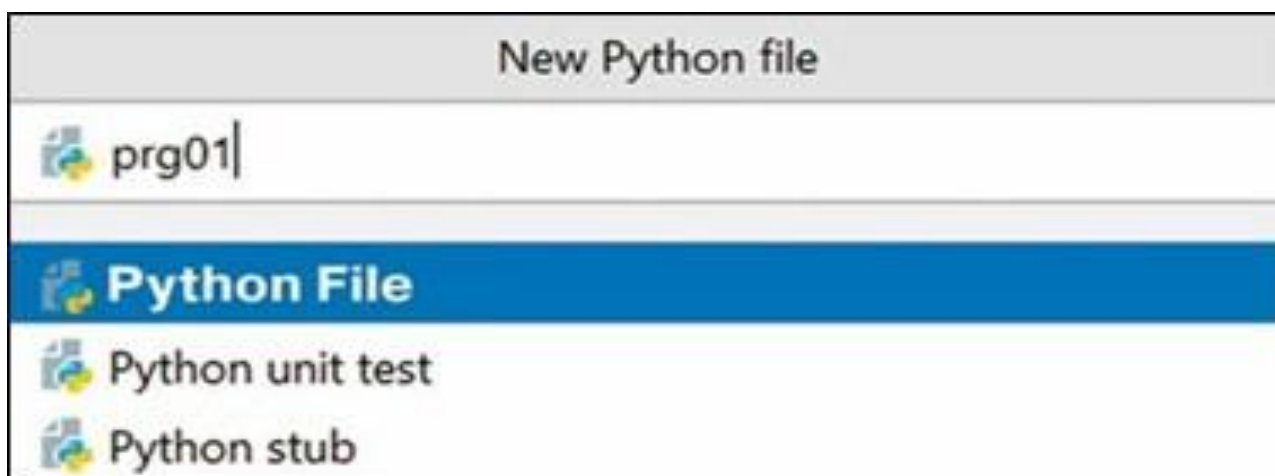


Рис. 11. Указание имени нового файла

Файл для написания программного кода создан, теперь потребуется установить библиотеки.

Библиотека (library) — сборник подпрограмм и объектов, используемых для разработки программного обеспечения.

Это уже написанные профессиональными программистами наборы проверенного кода. Готовые решения, которые мы можем вставлять в свой код и использовать. Нет необходимости каждый раз писать все алгоритмы с нуля, потому что всё это уже существует.

Библиотеки для программирования бывают встроенными и дополнительными.

Нам нужно добавить в проект дополнительную библиотеку: `dji Tello`. О ней нужно сказать несколько слов.

Библиотека, как это можно заметить из названия, содержит команды управления дроном Tello. Именно она позволяет дрону летать.

Нажмите `File Settings` и установите пакеты с необходимыми библиотеками (рис. 12,13). Используйте строку поиска и при указании библиотеки не забывайте нажимать «`Install Package`» (установить пакет).

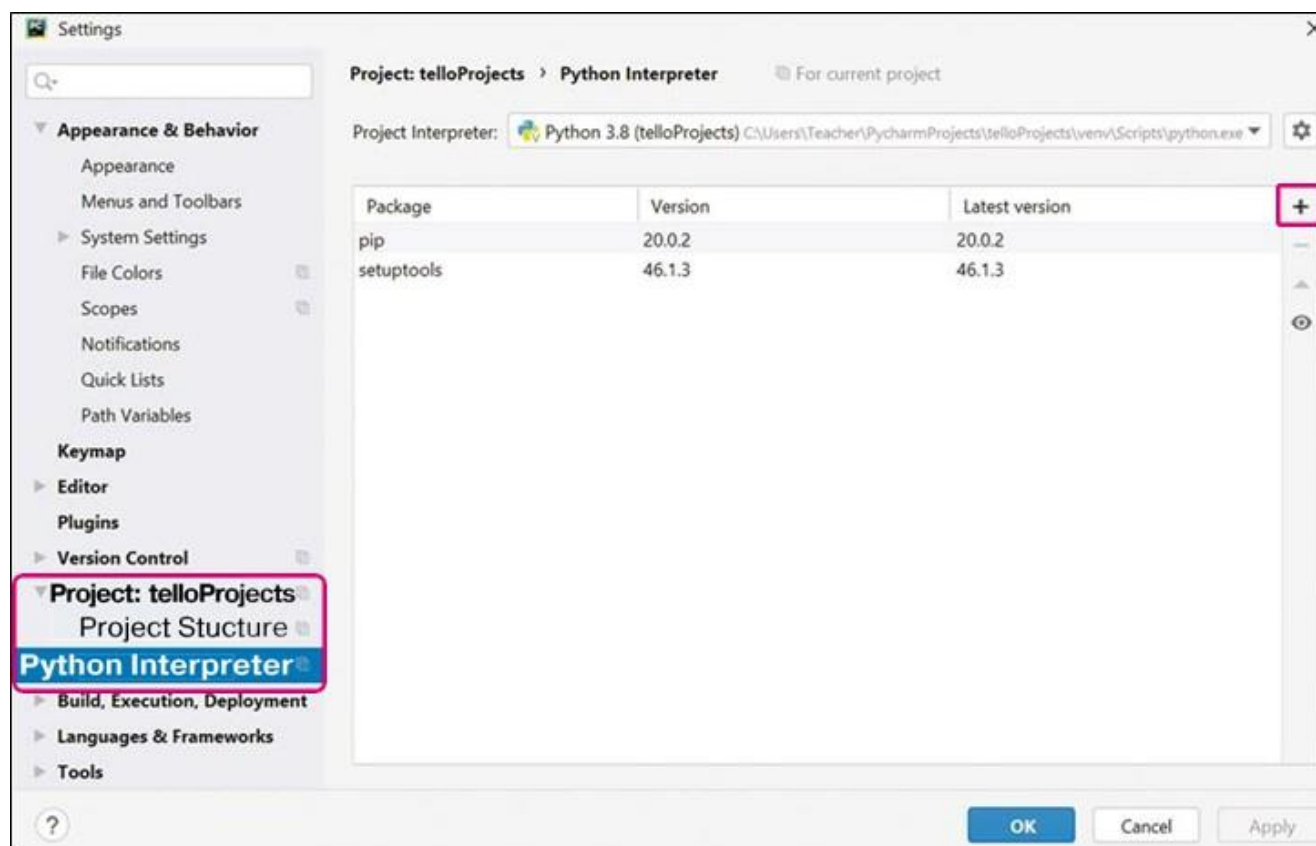


Рис. 12. Окно настроек. Настройки интерпретатора (нажать +)



Рис. 13. Поиск пакетов (библиотека dji Tellopy)

После выбора библиотеки нажимайте кнопку установки пакета. Далее закройте окно установки пакетов (используйте «крестик» в интерфейсе) и нажмите «ОК».

PyCharm разными способами поможет понять, что пакеты с библиотекой установлены (рис. 14).

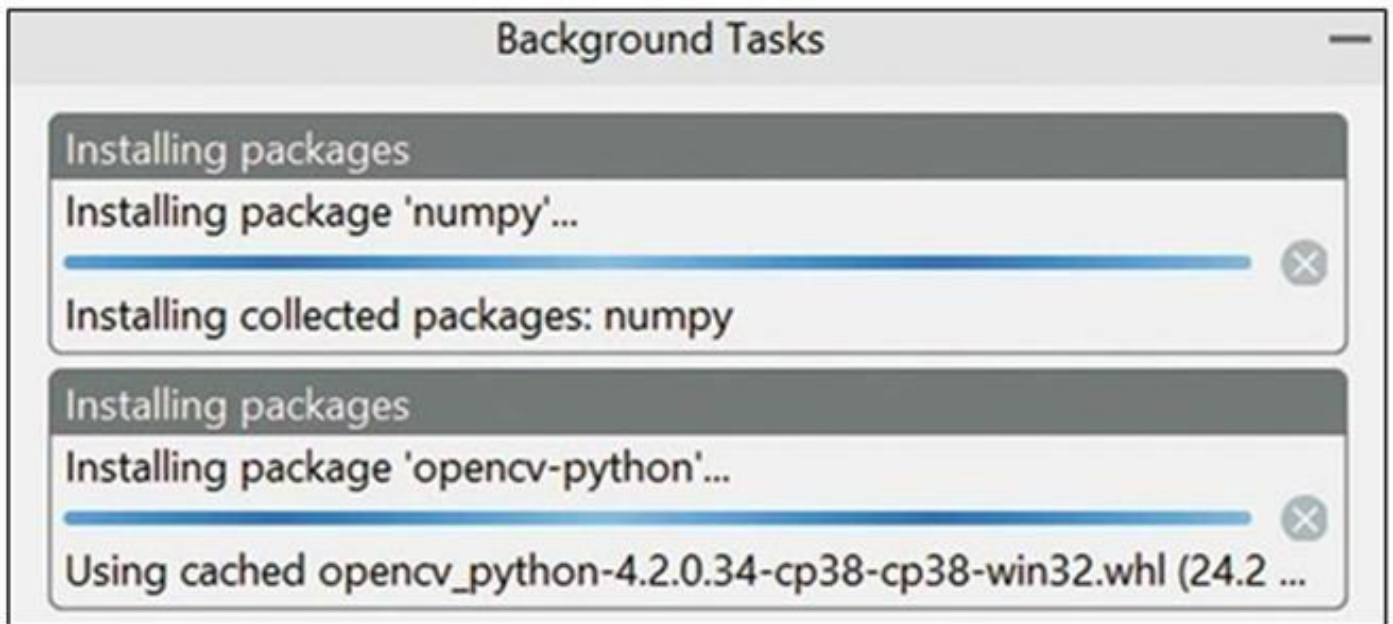


Рис. 14. Всплывающее окно с процессом установки пакетов

Давайте проверим, как установилась библиотек. Напишем программу, запустим её и посмотрим, что в консоли. Консоль — это окно для вывода системных сообщений и приёма команд.

Напишите программу (в файле `prg01.py`), подключающую наши три библиотеки и выводящую в консоль «Checking libraries...» (рис.15).

```
#подключение библиотек и проверка  
import cv2  
import numpy  
import djitellopy  
print ("Checking libraries...")
```

Рис. 15. Фрагмент программного кода

Если программа выполнена успешно, в консоли появится «Checking libraries...» (рис. 16). Ошибок не было, следовательно, библиотека работает. Мы же их импортировали в проект соответствующей командой (`import`).

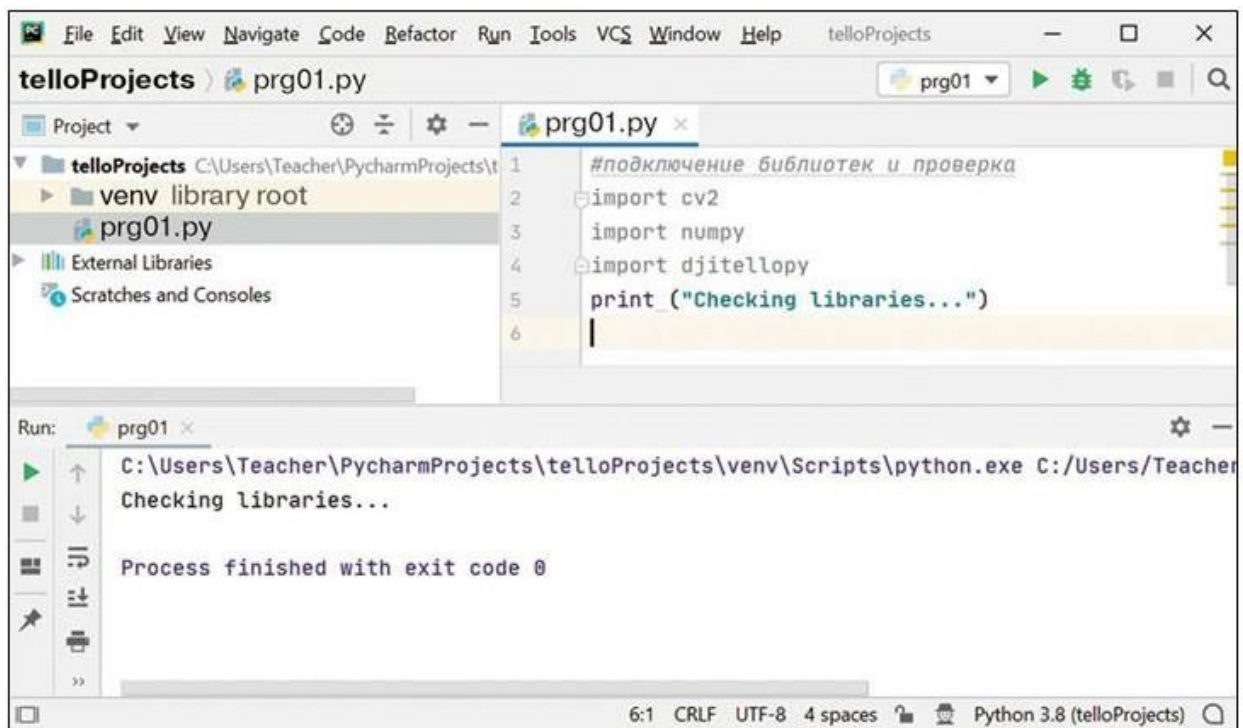


Рис. 16. Окно проекта TelloProjects

1.3 Первый запуск Tello

Конечно, вы знаете, как подключать гаджеты по Wi-Fi. Однако нужно повторить.

Если дронов несколько, как в школьном классе, то посмотрите сначала на вашем дроне наклейку с символьным названием беспроводной точки доступа Wi-Fi — SSID (Service Set Identifier). Она находится под аккумулятором. Далее — включить Tello. Нажать кнопку и подождать 10 секунд.

Следующий шаг — вызвать на панели задач меню выбора подключений к беспроводным сетям и выбрать SSID вашего дрона (рис. 17).

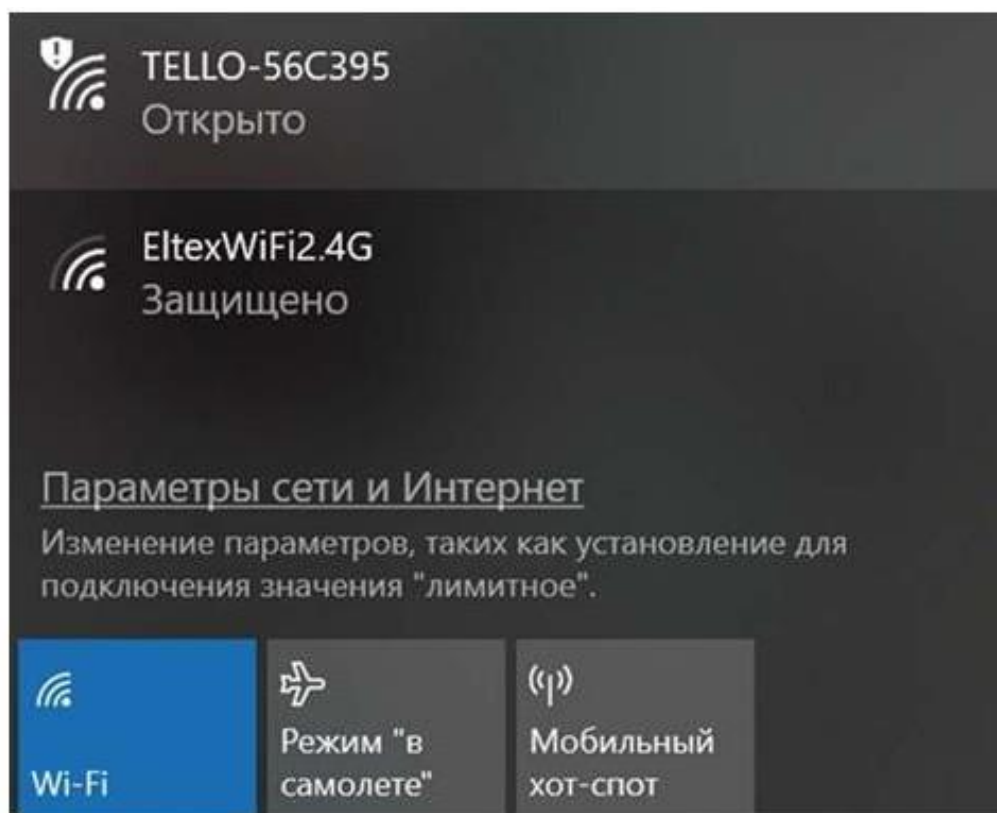


Рис. 17. Выбор точки доступа

После подключения к дрону на значке подключения появится восклицательный знак. Так и должно быть. Ваш коптер не раздаёт Интернет, он просто является точкой доступа к самому себе.

Подключите компьютер к Tello и обратите внимание, через сколько минут дрон самостоятельно выключится.

Пора проверить коммуникацию с дроном в PyCharm.

Запускайте среду, открывайте ваш проект, создавайте новый файл с кодом (prg02.py) и напишите небольшую программу, которая позволит проверить связь с Tello, а также ответит на очень неожиданные вопросы.

Изучите и проанализируйте программу, представленную на рисунке 18. Проверьте в работе несколько раз. Посмотрите, что выводится в консоли.

Несколько комментариев к программе.

— это знак, после которого пишется комментарий. Комментарии помогают понять алгоритм работы программы. На программу комментарии не влияют и их можно не писать.

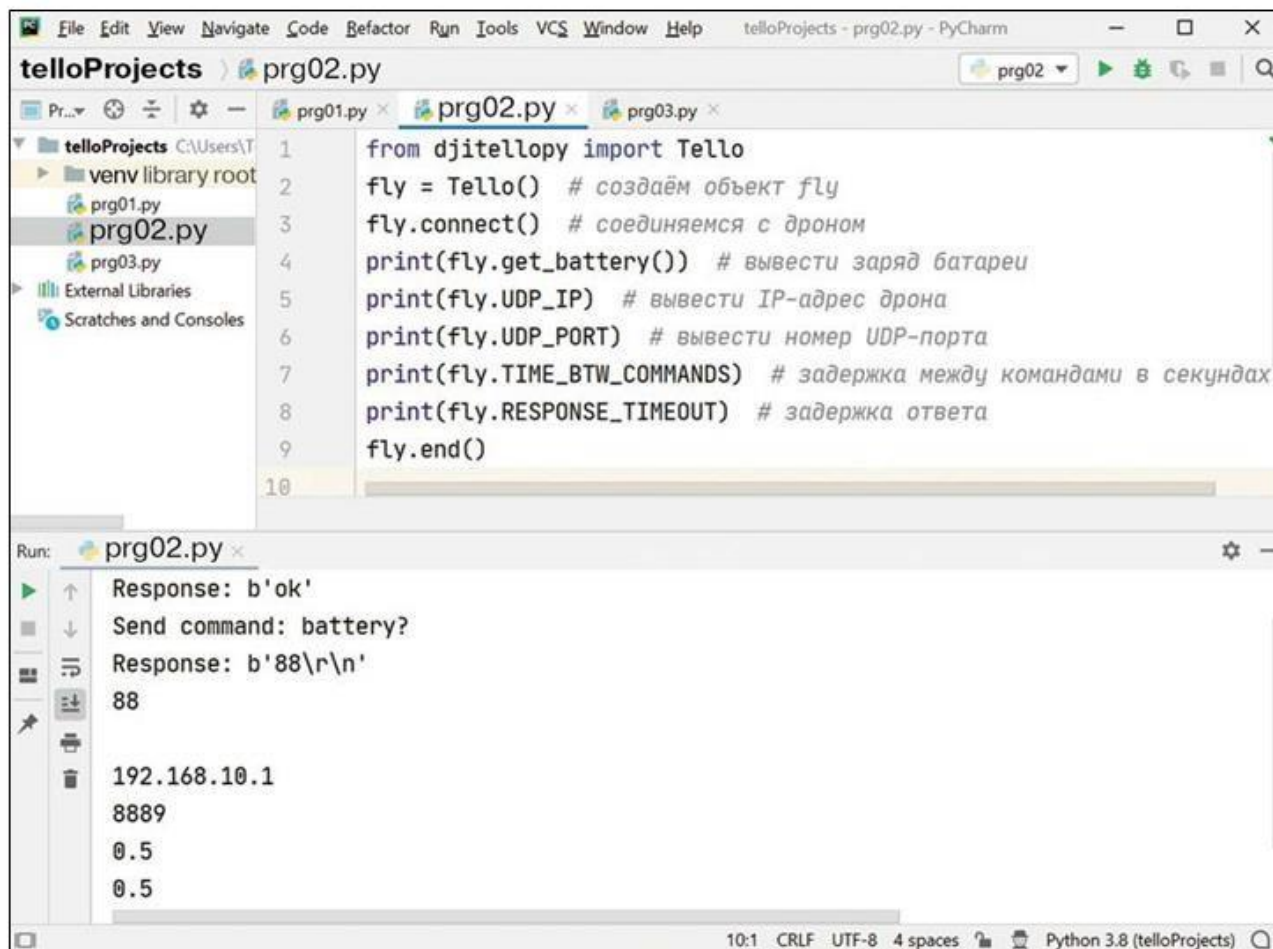


Рис. 18. Первая программа взаимодействия с дроном

В первой строке мы импортируем класс объектов Tello. Следующей командой мы говорим, что наш объект fly принадлежит к классу дронов Tello. Следовательно, ему становятся доступны все функции, методы и свойства объектов этого класса.

Обратите внимание на два параметра: задержка между командами и задержка ответа. Другими словами, получается, что между командами в программе должна быть задержка минимум в 1 с.

Вы обратили внимание, что, как только вы написали «fly.», появился список всех функций (f), свойств (p) и методов (m), доступных этому объекту (рис. 19)? Это

не просто очень удобная подсказка. Список позволит вам искать дополнительную информацию.



Рис. 19. Первая программа взаимодействия с дроном

Как вы поняли, знание английского языка абсолютно необходимо — на нём написана вся документация. Кроме того, технологии меняются очень быстро, и переводы за ними категорически не успевают.

Можно освоить английский в процессе написания программ и постоянного поиска решения проблем на иностранных форумах.

Например, чтобы подробнее прочитать об используемой библиотеке `dji Tello`, вы можете зайти в настройки, открыть список пакетов, как показывалось ранее, и перейти по указанной ссылке (рис. 20) на GitHub — крупнейший веб-сервис для хостинга IT-проектов и их совместной разработки.


damiafuentes Update README.md
 Latest commit 60c4a95 22 days ago

 djitellopy	dont use f"" string formatting as it is not supported pre python 3.6	4 months ago
 .gitignore	Initial commit	17 months ago
 LICENSE.txt	Before uploading to PyPi	17 months ago
 README.md	Update README.md	22 days ago
 example.py	fix the djitellopy import in example.py	4 months ago
 requirements.txt	Control the drone using pygame, python 2.7 and 3.6 compatibility	17 months ago
 setup.cfg	Before uploading to PyPi	17 months ago
 setup.py	v_1.5	17 months ago

 README.md



DJITelloPy

DJI Tello drone python interface using the official Tello SDK and Tello EDU SDK. Yes, this library has been tested with the drone. Please see [example.py](#) for a working example controlling the drone as a remote controller with the keyboard and the video stream in a window.

Рис. 20. Библиотека dji Tellopy на GitHub

РАЗДЕЛ 2. КОМАНДЫ УПРАВЛЕНИЯ И АЛГОРИТМИЧЕСКИЕ КОНСТРУКЦИИ

2.1 Команды управления полётом

Напишем программу, тестирующую основные полётные команды Tello. Пусть дрон взлетит, переместится назад, вперёд, вправо, влево, повернёт по часовой стрелке на 90^0 , против часовой на 90^0 и приземлится.

Задание №1

Изучите и проанализируйте программу, представленную далее. Проверьте в работе несколько раз. Посмотрите, что выводится в консоли. Попробуйте уменьшить задержки так, чтобы дрон перестал правильно выполнять алгоритм. Посмотрите при этом информацию в консоли.

```
import time # подключение библиотеки для работы со временем
from djitellopy import Tello
fly = Tello() # создаём объект fly
fly.connect() # соединяемся с дроном
print(fly.get_battery()) # проверяем заряд батареи
fly.speed = 30 # скорость в %
fly.takeoff() # подъём дрона
time.sleep(8) # ожидание, 8 с

fly.move_back(30) # назад на 30 см
time.sleep(3) # ожидание
fly.move_forward(30) # вперёд на 30 см
time.sleep(3) # ожидание
fly.move_left(30) # влево на 30 см
time.sleep(3) # ожидание
fly.move_right(30) # вправо на 30 см
time.sleep(3) # ожидание
fly.rotate_counter_clockwise(90) # поворот налево на 90°
time.sleep(3) # ожидание
fly.rotate_clockwise(90) # поворот направо на 90°
time.sleep(3) # ожидание
fly.land() # приземление
fly.end()
```

Рис. 21. Фрагмент программного кода

В программе выставлена задержка между командами три секунды. Можно проверить, как дрон выполнит задание, если указать меньшие задержки. Напишите программу для перемещений дрона вверх и вниз.

Важно помнить, что некоторые функции, Tello заблокирует при заряде аккумулятора меньше 50 %. Перевороты (flip) тоже будут заблокированы.

Задание №2

Изучите и проанализируйте программу, представленную ниже (рис.22). Проверьте в работе несколько раз. Посмотрите, что выводится в консоли.

```
import time # подключение библиотеки для работы со временем
from djitellopy import Tello
fly = Tello() # создаём объект fly
fly.connect() # соединяемся с дроном
print(fly.get_battery()) # проверяем заряд батареи
fly.speed = 30 # скорость в %
fly.takeoff() # подъём дрона
time.sleep(8) # ожидание
fly.flip_forward() # кувырок вперёд
time.sleep(3) # ожидание
fly.flip_back() # кувырок назад
print(fly.get_flight_time()) # общая продолжительность полётов
time.sleep(3) # ожидание
fly.land() # приземление
time.sleep(3) # ожидание
fly.end()
```

Рис. 22. Фрагмент программного кода

Задание №3

Напишите программу, демонстрирующую перевороты дрона вправо и влево.

Переменные и циклы

Какие из первых ярко выраженных проблем, с которыми столкнулись, вы уже точно можете сформулировать?

Изменять задержки неудобно.

Дрон постоянно ошибается и не возвращается точно в квадрат старта.

Много однотипных повторяющихся команд. Можно и больше перечислить

Давайте попробуем рассмотреть некоторые из возникших интересных ситуаций. Для этого нам потребуются переменные и циклы.

Переменная — это название зарезервированного места в оперативной памяти, предназначенного для хранения значений. Когда мы создаём переменную, мы резервируем место в оперативной памяти.

Слева от знака «=» — имя переменной, справа — значение, присвоенное этой переменной.

В Python не нужно объявлять тип переменной вручную, как, например, в языке C++ или Pascal. Объявление происходит автоматически, при присваивании значения. Это динамическая типизация.

Интерпретатор, основываясь на типе данных переменной, выделяет необходимое количество памяти.

Вы уже знаете чуть ли не с начальной школы, что в тех случаях, когда нужно повторить что-нибудь заданное количество раз, используют циклы. А если ещё точнее — цикл `for`. Если переменная входит в последовательность элементов, то выполняются указанные действия.

`for переменная in последовательность: действие`

Цикл `for` предназначен для последовательного перебора, говорят «итерирования», элементов из какой-то последовательности. Рассмотрим строку:

`for count in range(5):`

Функция `range(5)` создаст список из пяти элементов: 0, 1, 2, 3, 4.

А переменная `count` по порядку примет значения из этого списка. Таким образом, команды в цикле повторятся 5 раз.

Задание №4

Напишите программу, чтобы дрон перемещался вперёд/назад 5 раз. Оцените ошибку при многократном повторении. Проанализируйте код программы. Обсудите.

Внесите коррективы так, чтобы отклонение Tello от первоначального квадрата не превышало размеры самого дрона. Проверьте при 10-кратном повторении движений.

```
import time # подключение библиотеки для работы со временем
from djitellopy import Tello
fly = Tello() # создаём объект fly
dt = 3 # переменная для хранения значения задержки
distanceCm = 40 # переменная для хранения значения расстояния
fly.connect() # соединяемся с дроном
fly.speed = 40 # скорость в %
fly.takeoff() # подъём дрона
time.sleep(3 * dt) # ожидание
for count in range(5): # повторить 5 раз, count – счётчик цикла
    fly.move_back(distanceCm) # назад
    time.sleep(dt) # ожидание
    fly.move_forward(distanceCm+4) # вперёд
    time.sleep(dt) # ожидание
    print(count) # первое значение – 0, второе – 1, далее: 3, 4
fly.land() # приземление
time.sleep(dt) # ожидание
print(fly.get_battery()) # выводим заряд батареи
fly.end()
```

Рис. 23. Фрагмент программного кода

Задание №5

Используя циклы, протестируйте точность поворотов дрона вправо и влево. Протестируйте развороты. Обсудите. Подберите параметры команд так, чтобы ваш дрон после восьми повторений поворота вправо на 900 практически оставался в первоначальном положении.

Пора начинать самостоятельно составлять программы управления дроном Tello. Для этого можно использовать поле «Квадраты», а можно всё выполнить и без него.

Задание №6

Составьте алгоритм, чтобы дрон пролетел так, как показано на рисунке 24.

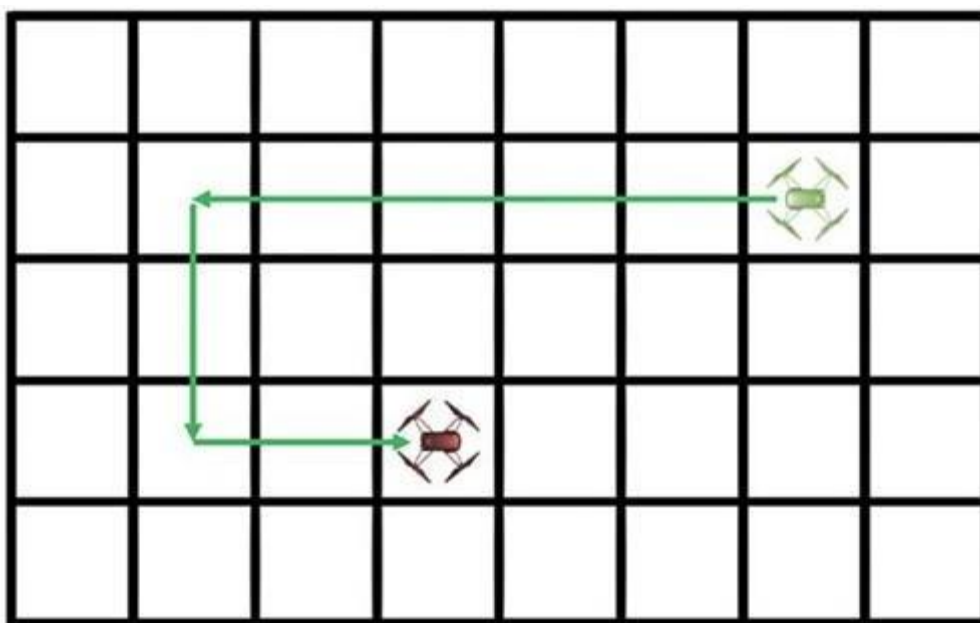


Рис. 24. Схема задания

Задание №7

Составьте программу, чтобы траектория движения Tello представляла собой прямоугольник (рис. 25).

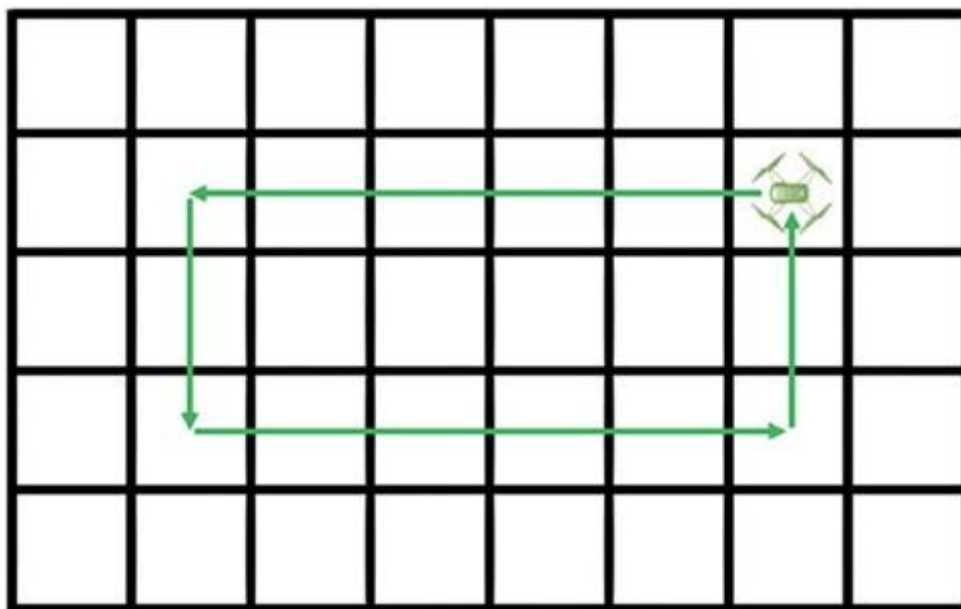


Рис. 25. Схема задания «Прямоугольник»

Задание №8

Составьте две программы, чтобы робот пролетал по квадрату с любой стороной (рис. 26). Используйте циклы.

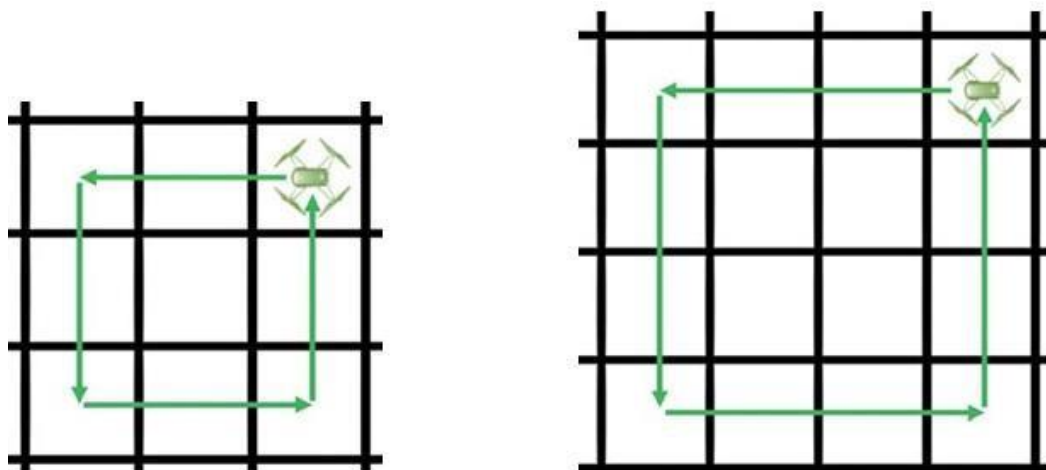


Рис. 26. Схемы заданий «Квадрат»

Задание №9

Составьте программу, чтобы робот описал в воздухе треугольник (рис. 27). Используйте циклы.

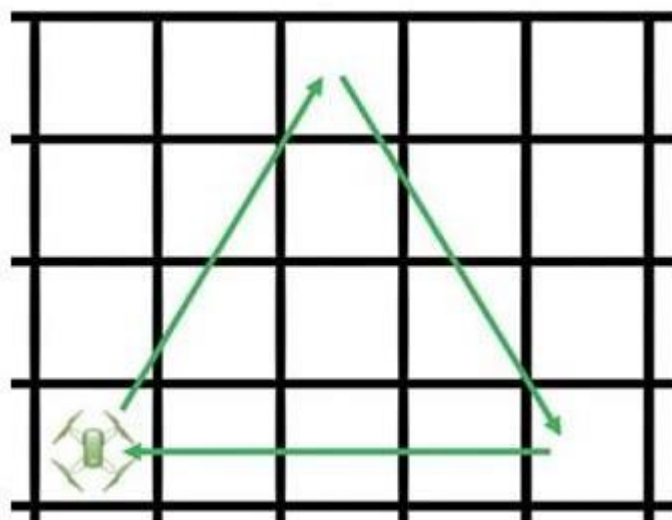


Рис. 27. Схема задания «Треугольник»

2.2 Ввод данных, цикл while, оператор if и функции

При выполнении задания по полёту вдоль сторон квадрата у вас должно было возникнуть желание написать универсальную программу: вводим с клавиатуры длину стороны квадрата, а дрон дальше сам всё делает.

Ввод информации с клавиатуры осуществляется с помощью функции `input()`. Она приостанавливает работу программы и ждёт ввод данных. После ввода нужно нажать клавишу «Enter», тогда программа продолжит работу.

Вводить данные в PyCharm можно в консоли (рис. 28).

Кроме самих данных, хорошо бы подумать о защите от некорректного ввода данных. Например, проверять, чтобы числа вводились из определённого числового диапазона.

Для этого нам потребуется цикл `while`. Этот вид циклов также используется для повторения частей кода, он выполняет работу до тех пор, пока верно определённое условие.

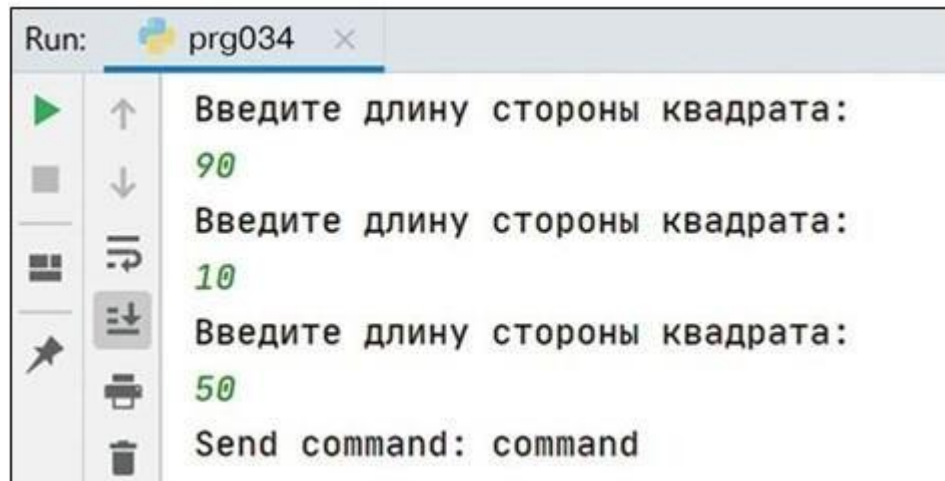


Рис. 28. Пример ввода данных

Задание №10

Изучите программу, представленную ниже. Исследуйте её работу. При необходимости дополнительной информации используйте поисковые запросы и возможности Интернета.

```

dist = 0
while dist < 20 or dist > 60:
    # ждём, пока не будет введено число из интервала (20, 60)
    print("Введите длину стороны квадрата: ")
    dist = int(input()) # вводим длину стороны
print("Полетели!")

```

Рис. 29. Фрагмент программного кода

Задание №11

Напишите программу, допускающую ввод с клавиатуры только целых чисел в интервале [20, 60]. Исследуйте. Проверьте в работе. Обсудите.

```

def is_int(a): # определяем функцию is_int
    try: # пытаемся...
        int(a) # ...преобразовать строку a в целое число
        return 1 # если прошло успешно, то функция возвращает 1
    except ValueError: # если ошибка, то...
        return 0 # функция возвращает 0

# после определения функции нужно оставить две пустые строки
dist = 0 # инициализация переменной
while dist < 20 or dist > 60:
    # ждём, пока не будет введено число из интервала (20, 60)
    print("Введите длину стороны квадрата: ")
    tmp = input() # вводим строку символов
    print(is_int(tmp)) # печатаем результат проверки
    if is_int(tmp) == 1: # если целое число
        dist = int(tmp) # преобразуем введённые символы в число
    print(dist) # выводим для контроля значения
print("Полетели!", dist, " см") # проверка закончена

```

Рис. 30. Фрагмент программного кода

Для проверки того, что число является целым, нужно написать небольшую функцию. Назовём её `is_int`. Функция в Python — это объект, принимающий аргументы и возвращающий значение. Функции определяются с помощью инструкции `def`.

Нашей функции мы «даём» строку, которую будем вводить с клавиатуры, а она должна проверить (try) возможность применения операции int (преобразование строки в целое число). Если всё хорошо, то результатом функции будет 1.

А если такое преобразование вызовет ошибку (except — исключение), то функция вернёт 0, то есть в результате будет 0.

Для обработки исключений/проблем используется конструкция try — except. В блоке try мы выполняем инструкцию, которая может породить исключения, а в блоке except — перехватываем их.

Итак, сначала обнуляем переменную, в которой будем хранить введённое число, но только число.

То, что мы введём с клавиатуры, запишем в переменную tmp.

Если введённая строка символов проходит проверку созданной функцией in_int, то только тогда мы записываем значение в переменную dist. Для этого используем условный оператор if. Он выбирает, какое действие следует выполнить в зависимости от значения переменных в момент проверки условия. В нашем случае мы пока использовали неполный условный оператор, то есть действия выполняются, только когда верно условие, если условие неверно — действий нет.

В условии цикла мы проверяем, чтобы число было в интервале от 20 до 60 включительно.

Обратите внимание, что для отладки программы используется вывод данных (print), чтобы можно было контролировать, что происходит в теле цикла while. Когда цикл while будет завершён, выведем введённое число с небольшим антуражем.

Теперь можно написать универсальную программу полёта дрона по квадрату, при этом длина стороны вводится пользователем.

Задание №13

Составьте универсальную программу, чтобы дрон пролетал по квадрату с любой стороной. Протестируйте и проанализируйте работу. Обсудите, что вызывает у вас сложности. При необходимости найдите дополнительную информацию в Сети.

```

import time # подключение библиотеки для работы со временем
from djitellopy import Tello

# перед определением функции нужны две пустые строки
def is_int(a): # определяем функцию is_int
    try: # пытаемся...
        int(a) # ...преобразовать строку a в целое число
        return 1 # если прошло успешно, то функция возвращает 1
    except ValueError: # если ошибка, то...
        return 0 # функция возвращает 0

# после определения функции нужны тоже две пустые строки
fly = Tello() # создаём объект fly, принадлежащий классу дронов
dt = 3 # значение задержки
dist = 0 # для хранения введённого числа
while dist < 20 or dist > 60:
    # ждём, пока не будет введено число из интервала (20, 60)
    print("Введите длину стороны квадрата: ")
    tmp = input() # вводим строку символов
    if is_int(tmp) == 1: # если целое число,
        dist = int(tmp) # тогда преобразуем строку в число
fly.connect() # соединяемся с дроном
fly.speed = 40 # скорость в %
fly.takeoff() # подъём дрона
time.sleep(3 * dt) # ожидание
for j in range(4): # повторить 4 раза
    fly.move_forward(dist) # вперёд
    time.sleep(dt) # ожидание
    fly.rotate_counter_clockwise(90) # поворот налево
    time.sleep(dt) # ожидание
fly.land() # приземление
print(fly.get_battery()) # выводим заряд батареи
fly.end()

```

Рис. 31. Фрагмент программного кода

Задание №14

Составьте универсальную программу, чтобы Tello пролетал по правильному треугольнику.

Задание №15

Составьте универсальную программу, чтобы дрон описывал следующие фигуры: пятиугольник, шестиугольник (рис. 32).

Если у вас есть интерес, можете самостоятельно проверить, появились ли такие возможности у Tello сейчас. Всё-таки выходят новые прошивки для дронов. В них могут возникать новые команды, новые возможности.

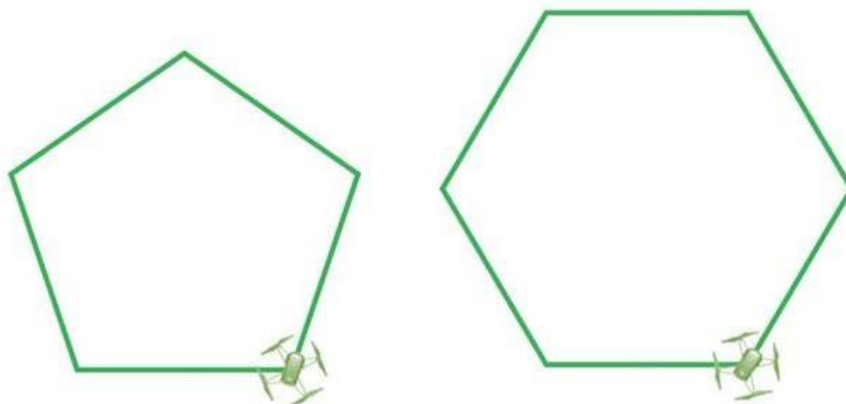


Рис. 32. Схема задания «Пятиугольник, шестиугольник»

2.3 Вложенные циклы

Давайте посмотрим на следующую схему полётного задания (рис. 33). Назовём её «Восьмёрка».

Одни циклы могут содержать внутри себя другие циклы — вложенные — это те циклы, которые размещены в теле других. В каждом цикле возможен свой вложенный цикл. Такую лестницу можно строить до бесконечности. Внешний цикл на первой итерации обращается с вызовом к внутреннему циклу, который выполняется до своего завершения. Затем управление перенаправляется на тело внешнего цикла.

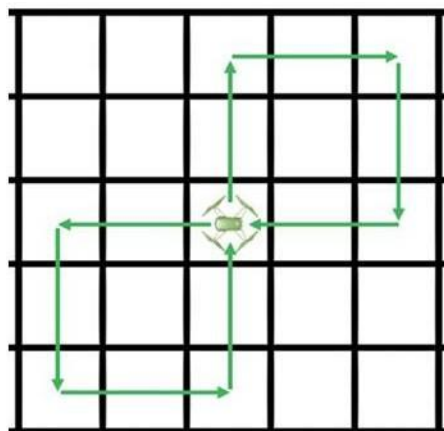


Рис. 33. Схема задания «Восьмёрка»

Задание №16

Изучите и проанализируйте программу, по которой дрон описывает восьмёрку (см. рис. 34). Протестируйте.

```
import time # подключение библиотеки для работы со временем
from djitellopy import Tello
fly = Tello() # создаём объект fly
dt = 3 # значение задержки
distance_cm = 50 # значение расстояния
fly.connect() # соединяемся с дроном
fly.speed = 40 # скорость в %
fly.takeoff() # подъём дрона
time.sleep(3 * dt) # ожидание
for i in range(2): # повторить 2 раза (два квадрата)
    for j in range(4): # повторить 4 раза (сторона и угол)
        fly.move_forward(distance_cm) # вперёд
        time.sleep(dt) # ожидание
        if j < 3: # нужно только три поворота
            fly.rotate_counter_clockwise(90) # поворот налево
            time.sleep(dt) # ожидание
        fly.rotate_clockwise(90) # поворот направо
        time.sleep(dt) # ожидание
fly.land() # приземление
print(fly.get_battery()) # выводим заряд батареи
fly.end()
```

Рис. 34. Программный код

Задание №17

Составьте программу, чтобы дрон описывал восьмёрку, но как показано на рисунке 35. Протестируйте. Проанализируйте работу.

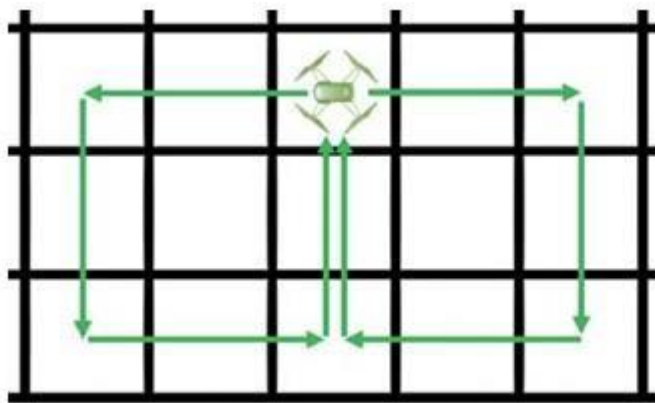


Рис. 35. Схема задания

Задание №18

Придумайте свою траекторию, чтобы при выполнении задания дроном использовались вложенные циклы.

2.4 Угловое положение

Во время полёта дрон всегда движется по сложной траектории в пространстве. Для описания этого движения вводят оси, связанные с дроном, и рассматривают их движение относительно другой системы координат, например привычной декартовой (но на земле). Такая система координат движется и вращается вместе с дроном.

Одна из систем координат, используемых для анализа движения воздушных судов в полёте, — связанная система координат. Она состоит из продольной оси OX , поперечной оси OZ и вертикальной оси OY , которые проходят через центр масс движущегося объекта (рис. 28).

Движение при вращении вокруг продольной оси (OX) называется креном (roll), вокруг вертикальной оси (OY) — рысканием (yaw), а вокруг поперечной оси (OZ) — тангажом (pitch).

Углы тангажа, крена и рыскания (в Tello используется термин «attitude» — угловое положение) однозначно задают ориентацию в пространстве. Иногда слово «угол» опускают и говорят просто «тангаж», «крен», «рыскание».

В любом квадрокоптере имеется IMU- сенсор. IMU — это Inertial Measurement Unit — инерционное измерительное устройство. Именно оно позволяет определить положение в пространстве.

Действие сенсора основано на многоосевой комбинации гироскопа, акселерометра, магнетометра и датчика давления (они внутри IMU-сенсора). Данные с этих сенсоров обрабатываются достаточно сложными алгоритмами и преобразуются в углы крена, тангажа и рыскания. Если хотите глубоко погрузиться в этот вопрос, то следующее задание как раз подойдёт.



Рис. 36. Система координат

Задание №19

Изучите и проанализируйте программу, выводящую в консоль следующие значения: углы тангажа, крена, рыскания дрона, показание барометра, расстояние до пола, заряд аккумуляторной батареи. Протестируйте. Дрон фиксируйте в руках чётко.

```

print(fly.get_battery()) # проверяем заряд батареи
time.sleep(dt)
print(fly.get_attitude()) # углы тангажа, крена, рыскания
time.sleep(dt)
print(fly.get_barometer()) # вывести значения барометра
time.sleep(dt)
print(fly.get_distance_tof()) # расстояние до пола
time.sleep(2*dt)
i = i + 1 # увеличиваем значение переменной на 1
fly.end()

import time # включаем библиотеку работы со временем
from djitellopy import Tello
fly = Tello() # создаём объект fly
dt = 0.5 # задаём задержку между командами
i = 0 # задаём первоначальное значение переменной выхода из цикла
fly.connect() # соединяемся с дроном
while i < 5: # тело цикла выполняется, пока i < 5

```

Рис.36. Программный код

2.5 Система визуального позиционирования

Вы обратили внимание на метод `get_distance_tof()`? Метод в программировании — это функция, принадлежащая какому-то классу (в нашем случае классу `Tello`) или объекту (у нас объект `fly`).

Первые два слова вы перевели, а что такое TOF?

Вы знаете, что в `Tello` используется система визуального позиционирования: камера и инфракрасный объёмный датчик (рис.29).

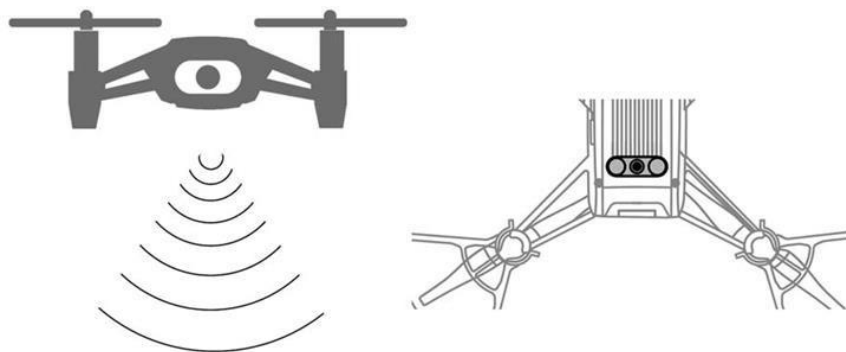


Рис.37. Система визуального позиционирования

TOF, или полностью Time of Flight, переводится как «время полёта». Фактически это датчик, который излучает инфракрасные лучи для измерения глубины. Да, снизу дрон строит объёмную карту! Конечно, с небольшим разрешением (в пикселях), но каждая точка этого «снимка» содержит информацию о расстоянии до дрона.

TOF-камеры ещё называют камерами глубины, снимающими видео, в каждом пикселе которого хранится не цвет, а расстояние до объекта в этой точке. Принцип работы камеры такой:

с помощью собственного сенсора камера излучает инфракрасный импульс, который отражается от объектов и возвращается обратно в камеру;

возвращённый импульс по-разному засвечивает пиксели на матрице камеры: чем ближе объект, тем ярче пиксель.

Таким образом, все объекты на изображении получаются объёмными, и формируется объёмная карта местности.

При старте дрон поднимается на высоту более 50 см, а лететь рекомендуется выше 1 метра.

Скорость работы процессоров велика, и скорость света по сравнению с ней уже не выглядит впечатляющей. За один такт процессора на 600 МГц (он в Tello установлен) свет успеет «пролететь» 50 см. Получается два такта на 1 м. Следовательно, летать ближе 50 см нельзя...

А для того, чтобы надёжно определять объекты, находящиеся ближе 50 см, в Tello используется обычный ИК-дальномер. Это излучатель, который вы можете видеть, как только включится дрон (он красным светится). Вот поэтому и называется система визуального позиционирования «два сенсора и сложные алгоритмы обработки изображений».

Задание №20

Подкорректируйте предыдущую программу. Внимательно исследуйте все показания. Дрон фиксируйте в руках чётко, не размахивайте, не трясите. Помните, что, если дрон не летает, значит, нарушено его охлаждение! Делайте всё достаточно быстро. Проанализируйте точность работы на разной высоте. Обязательно всё обсудите.